Week 4: Database Management System Implementation

Name: Sirisha Kummari

Matriculation number: 72415466

Introduction:

The objective of this assignment is to design and implement a bookstore database using PostgreSQL. The system will manage information about:

- Books: Title, author, ISBN, price, stock quantity
- Customers: Name, contact details, and student ID (for student customers)
- Orders: Order date, customer details, total amount
- Order Items: Books in each order, quantity, and price at the time of purchase

The database will also support several operational queries, including:

- Retrieving books currently in stock
- Updating stock after a sale
- Calculating total sales for the current month
- Retrieving all orders placed by a specific customer

This project demonstrates how to create a simple database system and use SQL queries to handle basic bookstore operations.

Part 1: Database Setup:

Install PostgreSQL and pgAdmin:

To begin the assignment, I installed PostgreSQL and pgAdmin on my system by following the official installation guides.

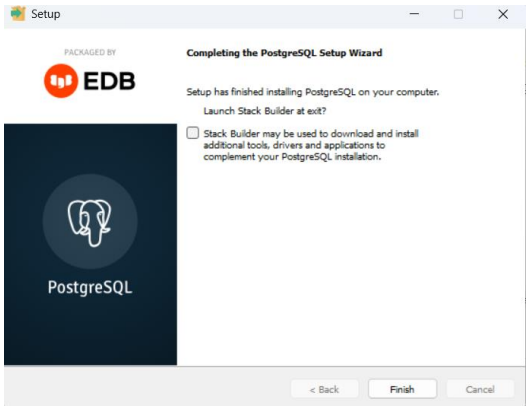
- PostgreSQL Version: 18
- pgAdmin Version: 9.8

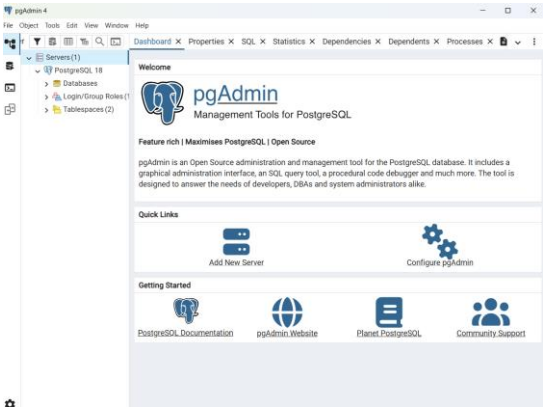
Installation Steps:

1. Downloaded the PostgreSQL installer from the official website:
    - PostgreSQL Download
    - Windows Installer
    - EnterpriseDB Installer
2. Ran the installer and followed the step-by-step instructions to install PostgreSQL.
3. pgAdmin was automatically set up with PostgreSQL.

4. Opened pgAdmin, connected to the PostgreSQL server, and confirmed everything was working properly.

Both PostgreSQL and pgAdmin were successfully installed and configured.



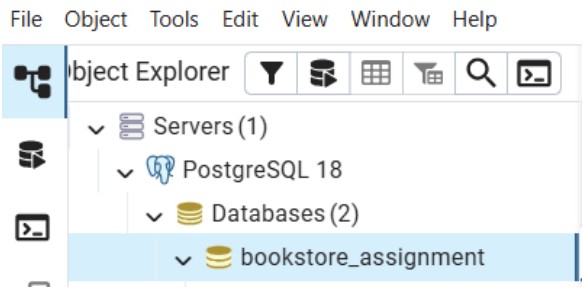


Create the Database:

Using pgAdmin, I created the database named bookstore_assignment by following these steps:

1. Opened pgAdmin and connected to the PostgreSQL server.
2. Right-clicked on the "Databases" node.
3. Selected the "Create" option and chose "Database".
4. In the "Create Database" window, entered bookstore_assignment as the database name and clicked "Save" to complete the creation process.

This process successfully created the database, which will be used to store the bookstore's data.

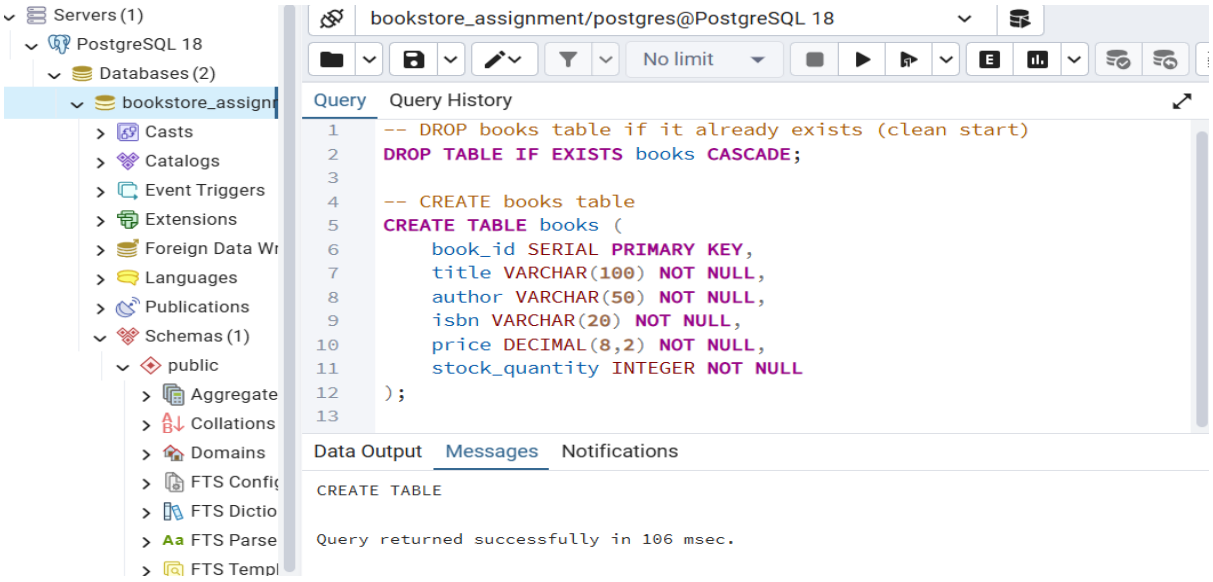


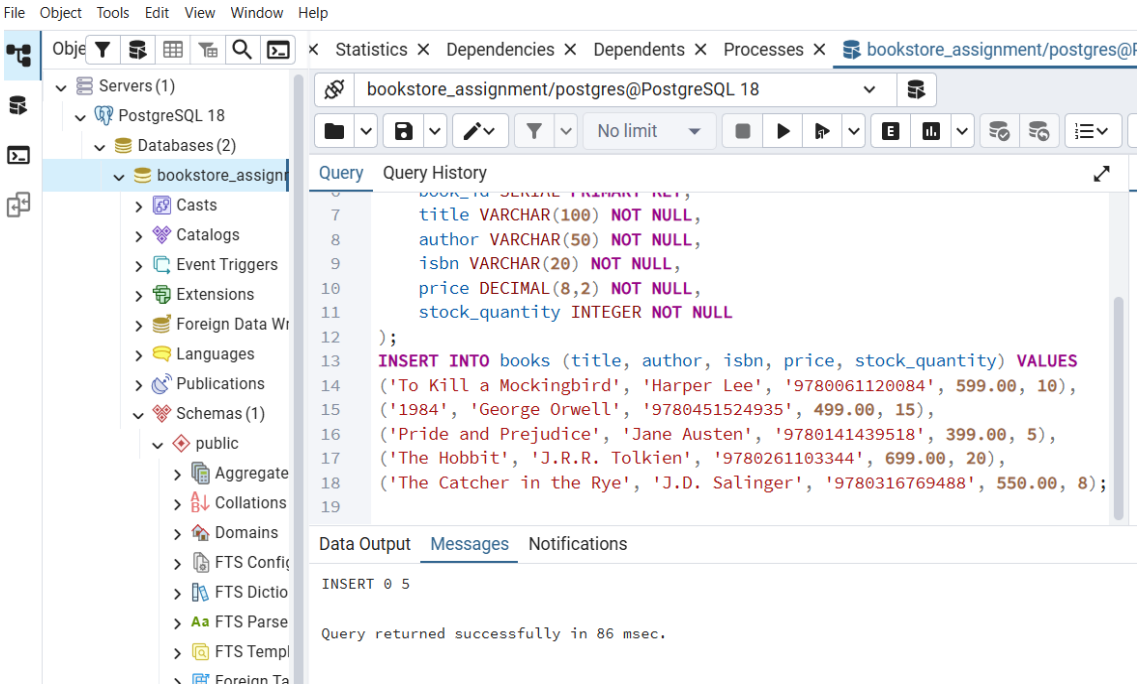
Part 2: Database Implementation

Step 1: Create Tables and Insert Data:

Create and Insert into the books Table

The books table was created to store details about the books in the bookstore, including the title, author, ISBN, price, and stock quantity. After creating the table, I inserted sample data for five books.





```
);
INSERT INTO books (title, author, isbn, price, stock_quantity) VALUES
('To Kill a Mockingbird', 'Harper Lee', '9780061120084', 599.00, 10),
('1984', 'George Orwell', '9780451524935', 499.00, 15),
('Pride and Prejudice', 'Jane Austen', '9780141439518', 399.00, 5),
('The Hobbit', 'J.R.R. Tolkien', '9780261103344', 699.00, 20),
('The Catcher in the Rye', 'J.D. Salinger', '9780316769488', 550.00, 8);
SELECT * FROM books;
SELECT
    book_id,
    title,
    author,
    isbn,
    '₹' || price AS price  -- adds ₹ before the number
```

	book_id [PK] integer	title character varying (100)	author character varying (50)	isbn character varying (20)	price text	stock_quantity integer
1	1	To Kill a Mockingbird	Harper Lee	9780061120084	₹599.00	10
2	2	1984	George Orwell	9780451524935	₹499.00	15
3	3	Pride and Prejudice	Jane Austen	9780141439518	₹399.00	5
4	4	The Hobbit	J.R.R. Tolkien	9780261103344	₹699.00	20
5	5	The Catcher in the Rye	J.D. Salinger	9780316769488	₹550.00	8

Create and Insert into the customers Table:

The customers table was created to store information about the bookstore's customers, including their name, contact details, customer type, and student ID (if applicable). I then inserted data for three sample customers.

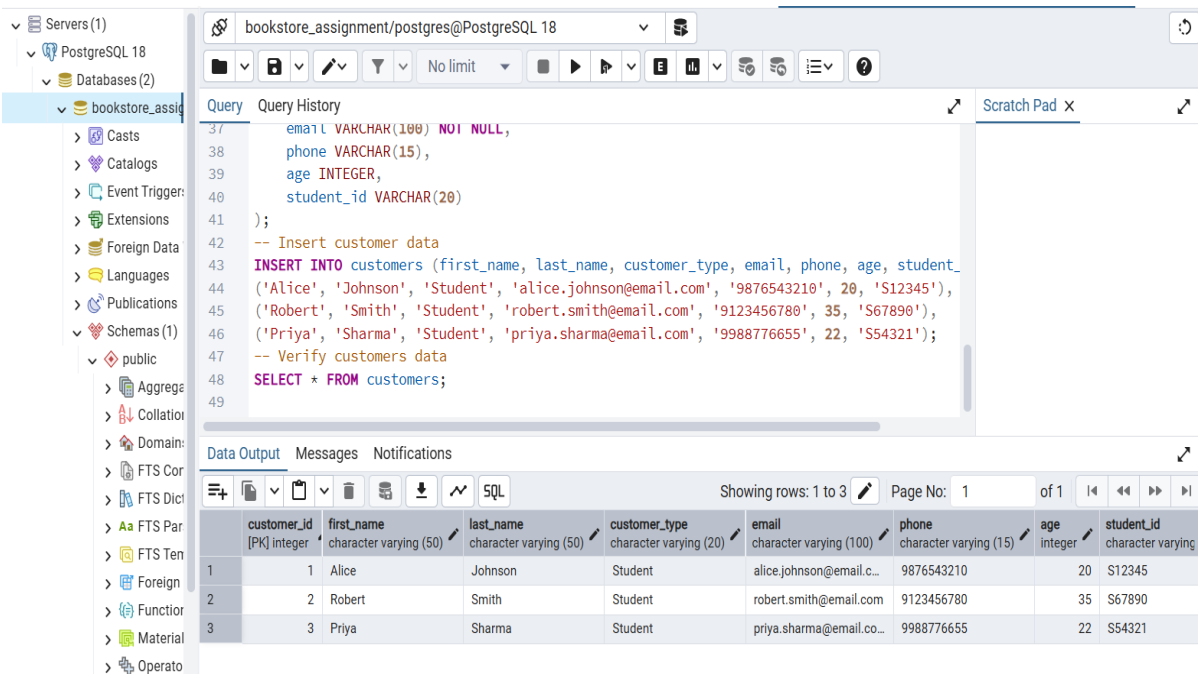
```
-- Create customers table
CREATE TABLE customers (
    customer_id SERIAL PRIMARY KEY,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    customer_type VARCHAR(20) NOT NULL,
    email VARCHAR(100) NOT NULL,
    phone VARCHAR(15),
    age INTEGER,
    student_id VARCHAR(20)
);
```

```
CREATE TABLE

Query returned successfully in 113 msec.
```

```
    phone VARCHAR(15),
    age INTEGER,
    student_id VARCHAR(20)
);
-- Insert customer data
INSERT INTO customers (first_name, last_name, customer_type, email, phone,
('Alice', 'Johnson', 'Student', 'alice.johnson@email.com', '9876543210', 20
('Robert', 'Smith', 'Student', 'robert.smith@email.com', '9123456780', 35,
('Priya', 'Sharma', 'Student', 'priya.sharma@email.com', '9988776655', 22,
```

Data Output | Messages | Notifications

```
INSERT 0 3

Query returned successfully in 168 msec.
```



Create and Insert into the orders Table

The orders table was created to store details about customer orders, including the customer ID, order date, and total amount. Sample data for two orders was then inserted.

Query | Query History

```sql
-- Verify customers data
SELECT * FROM customers;

-- Recreate orders table
CREATE TABLE orders (
    order_id SERIAL PRIMARY KEY,
    customer_id INTEGER REFERENCES customers(customer_id),
    order_date DATE NOT NULL,
    total_amount DECIMAL(10,2) NOT NULL
);
```

Data Output | Messages | Notifications

```
CREATE TABLE

Query returned successfully in 94 msec.
```

bookstore_assignment/postgres@PostgreSQL 18

No limit

Query | Query History

```sql
SELECT * FROM customers;

-- Recreate orders table
CREATE TABLE orders (
    order_id SERIAL PRIMARY KEY,
    customer_id INTEGER REFERENCES customers(customer_id),
    order_date DATE NOT NULL,
    total_amount DECIMAL(10,2) NOT NULL
);
-- Insert orders data
INSERT INTO orders (customer_id, order_date, total_amount) V
(1, '2025-10-10', 1198.00),
(3, '2025-10-12', 799.00);
```

Data Output | Messages | Notifications

```
INSERT 0 2

Query returned successfully in 91 msec.
```

Query | Query History

```sql
-- Recreate orders table
CREATE TABLE orders (
    order_id SERIAL PRIMARY KEY,
    customer_id INTEGER REFERENCES customers(customer_id),
    order_date DATE NOT NULL,
    total_amount DECIMAL(10,2) NOT NULL
);
-- Insert orders data
INSERT INTO orders (customer_id, order_date, total_amount) V
(1, '2025-10-10', 1198.00),
(3, '2025-10-12', 799.00);
-- Verify orders data
SELECT * FROM orders;
```

Data Output | Messages | Notifications

SQL | Showing rows: 1 to 2 | Page

	order_id [PK] integer	customer_id integer	order_date date	total_amount numeric (10,2)
1	1	1	2025-10-10	1198.00
2	2	3	2025-10-12	799.00

Create and Insert into the order_items Table

The order_items table was created to link books with orders, storing the quantity and price of each item. I inserted sample data linking specific books to the orders.

```
INSERT INTO orders (customer_id, order_date, total_amount) V
(1, '2025-10-10', 1198.00),
(3, '2025-10-12', 799.00);
-- Verify orders data
SELECT * FROM orders;
-- Recreate order_items table
CREATE TABLE order_items (
    order_item_id SERIAL PRIMARY KEY,
    order_id INTEGER REFERENCES orders(order_id),
    book_id INTEGER REFERENCES books(book_id),
    quantity INTEGER NOT NULL,
    price DECIMAL(8,2) NOT NULL
);
```

Data Output | Messages | Notifications

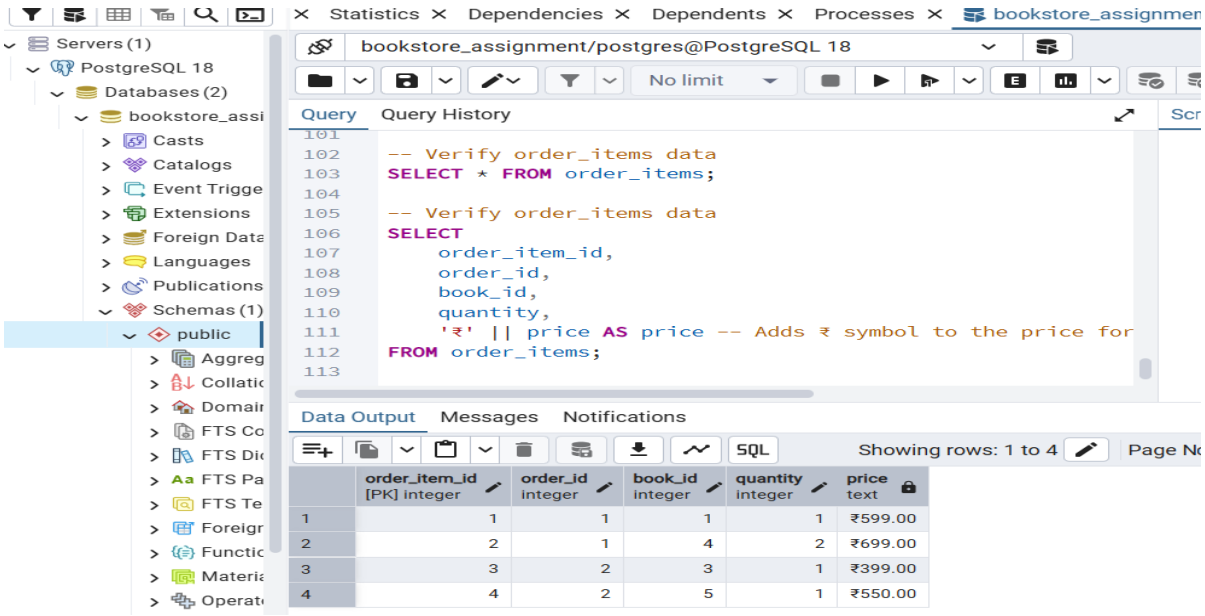
```
CREATE TABLE

Query returned successfully in 112 msec.
```

bookstore_assignment/postgres@PostgreSQL 18

```
    order_id INTEGER REFERENCES orders(order_id),
    book_id INTEGER REFERENCES books(book_id),
    quantity INTEGER NOT NULL,
    price DECIMAL(8,2) NOT NULL
);

-- Insert order items data (linking books to orders)
INSERT INTO order_items (order_id, book_id, quantity, price
(1, 1, 1, 599.00),   -- Order 1, Book 1 (1 quantity)
(1, 4, 2, 699.00),   -- Order 1, Book 4 (2 quantities)
(2, 3, 1, 399.00),   -- Order 2, Book 3 (1 quantity)
(2, 5, 1, 550.00);   -- Order 2, Book 5 (1 quantity)
```

Data Output | Messages | Notifications

```
INSERT 0 4

Query returned successfully in 86 msec.
```



SQL Codes of Part 2 & Part 3:

```
-- ==========================
-- PART 1: Drop Existing Tables (in correct order)
-- ==========================
-- Drop order_items table first (since it depends on orders)
DROP TABLE IF EXISTS order_items;

-- Drop orders table second (since it depends on customers)
DROP TABLE IF EXISTS orders;

-- Drop customers table third (no dependencies left after orders and order_items are dropped)
DROP TABLE IF EXISTS customers;

-- Drop books table last (no dependencies)
DROP TABLE IF EXISTS books;

-- ==========================
-- PART 2: Recreate Tables and Insert Data
-- ==========================

-- Recreate books table
CREATE TABLE books (
    book_id SERIAL PRIMARY KEY,
    title VARCHAR(100) NOT NULL,
    author VARCHAR(50) NOT NULL,
    isbn VARCHAR(20) NOT NULL,
    price DECIMAL(8,2) NOT NULL,
    stock_quantity INTEGER NOT NULL
);

-- Insert books data
INSERT INTO books (title, author, isbn, price, stock_quantity) VALUES
('To Kill a Mockingbird', 'Harper Lee', '9780061120084', 599.00, 10),
('1984', 'George Orwell', '9780451524935', 499.00, 15),
('Pride and Prejudice', 'Jane Austen', '9780141439518', 399.00, 5),
('The Hobbit', 'J.R.R. Tolkien', '9780261103344', 699.00, 20),
```

```
('The Catcher in the Rye', 'J.D. Salinger', '9780316769488', 550.00, 8);

-- Verify books data
SELECT * FROM books;

-- Show books with ₹ symbol in price
SELECT
    book_id,
    title,
    author,
    isbn,
    '₹' || price AS price,  -- Adds ₹ before the price
    stock_quantity
FROM books;

-- Recreate customers table
CREATE TABLE customers (
    customer_id SERIAL PRIMARY KEY,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    customer_type VARCHAR(20) NOT NULL,
    email VARCHAR(100) NOT NULL,
    phone VARCHAR(15),
    age INTEGER,
    student_id VARCHAR(20)
);

-- Insert customers data
INSERT INTO customers (first_name, last_name, customer_type, email, phone, age, student_id) VALUES
('Alice', 'Johnson', 'Student', 'alice.johnson@email.com', '9876543210', 20, 'S12345'),
('Robert', 'Smith', 'Student', 'robert.smith@email.com', '9123456780', 35, 'S67890'),
('Priya', 'Sharma', 'Student', 'priya.sharma@email.com', '9988776655', 22, 'S54321');

-- Verify customers data
SELECT * FROM customers;

-- Recreate orders table
CREATE TABLE orders (
    order_id SERIAL PRIMARY KEY,
    customer_id INTEGER REFERENCES customers(customer_id),
    order_date DATE NOT NULL,
    total_amount DECIMAL(10,2) NOT NULL
);
-- Insert orders data
INSERT INTO orders (customer_id, order_date, total_amount) VALUES
(1, '2025-10-10', 1198.00),
(3, '2025-10-12', 799.00);
-- Verify orders data
SELECT * FROM orders;
-- Recreate order_items table
CREATE TABLE order_items (
    order_item_id SERIAL PRIMARY KEY,
    order_id INTEGER REFERENCES orders(order_id),
    book_id INTEGER REFERENCES books(book_id),
    quantity INTEGER NOT NULL,
    price DECIMAL(8,2) NOT NULL
);

-- Insert order items data (linking books to orders)
INSERT INTO order_items (order_id, book_id, quantity, price) VALUES
```

```
(1, 1, 1, 599.00),  -- Order 1, Book 1 (1 quantity)
(1, 4, 2, 699.00),  -- Order 1, Book 4 (2 quantities)
(2, 3, 1, 399.00),  -- Order 2, Book 3 (1 quantity)
(2, 5, 1, 550.00);  -- Order 2, Book 5 (1 quantity)

-- Verify order_items data
SELECT * FROM order_items;

-- Verify order_items data
SELECT
    order_item_id,
    order_id,
    book_id,
    quantity,
    '₹' || price AS price -- Adds ₹ symbol to the price for display
FROM order_items;

-- ========================
-- PART 3: Operational Queries (Daily Operations)
-- ========================

-- 1. Show all books currently in stock
SELECT
    book_id,
    title,
    author,
    isbn,
    price,
    stock_quantity
FROM books
WHERE stock_quantity > 0;

-- 2. Find a customer by last name
SELECT
    customer_id,
    first_name,
    last_name,
    email,
    phone
FROM customers
WHERE last_name = 'Smith';  -- Replace 'Smith' with the last name you're searching for.

-- 3. Calculate total sales for the current month
SELECT
    SUM(total_amount) AS total_sales
FROM orders
WHERE EXTRACT(MONTH FROM order_date) = EXTRACT(MONTH FROM CURRENT_DATE)
  AND EXTRACT(YEAR FROM order_date) = EXTRACT(YEAR FROM CURRENT_DATE);

  -- 4. Update stock when a book is sold
UPDATE books
SET stock_quantity = stock_quantity - 1
WHERE book_id = 1;  -- Example: Decrease stock of book ID 1 by 1

-- 5. List all orders for a specific customer with price in ₹ symbol
SELECT
    o.order_id,
    o.order_date,
    o.total_amount,
    oi.book_id,
```

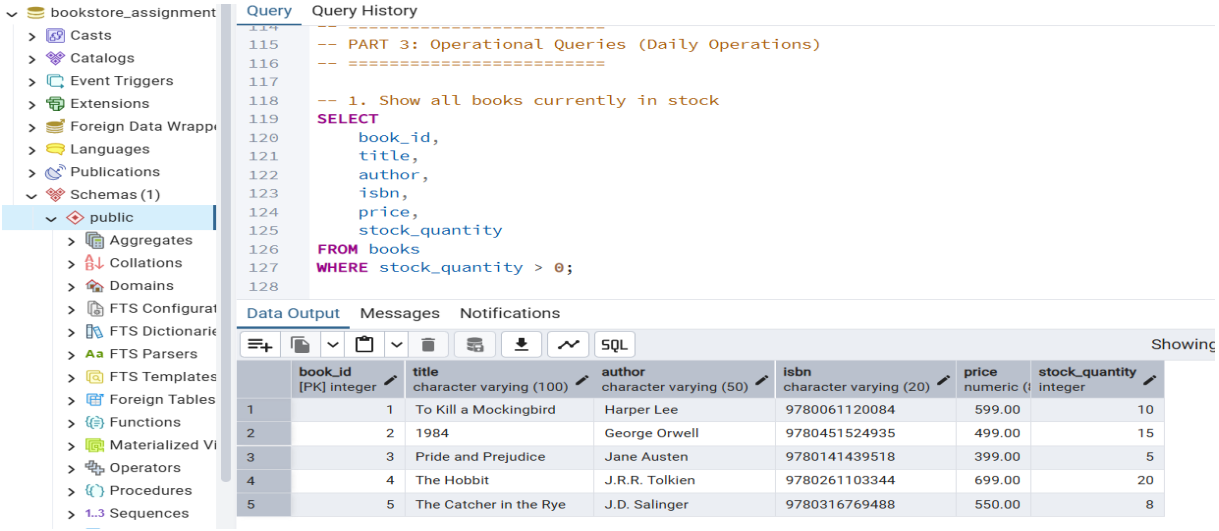
```
    oi.quantity,
    '₹' || oi.price AS price  -- Adds ₹ symbol before the price
FROM orders o
JOIN order_items oi ON o.order_id = oi.order_id
WHERE o.customer_id = 1;  -- Replace '1' with the customer_id you're interested in
```

Part 3: Operational Queries (Daily Operations):

In this section, I wrote SQL queries that simulate the daily operations a bookstore clerk would perform, such as checking stock, retrieving customer details, and calculating sales.

Step 1: Show All Books Currently in Stock

This query retrieves a list of all books that are currently in stock (i.e., having a stock quantity greater than 0).



Step 2: Find a Customer by Last Name

This query allows the clerk to find customer details by their last name. In this case, I searched for a customer with the last name "Smith".

Query Query History

```
    stock_quantity
FROM books
WHERE stock_quantity > 0;

-- 2. Find a customer by last name
SELECT
    customer_id,
    first_name,
    last_name,
    email,
    phone
FROM customers
WHERE last_name = 'Smith';  -- Replace 'Smith' with the last name you're searching for.
```

Data Output Messages Notifications

Showing rows: 1 to 1 Page No: 1

	customer_id [PK] integer	first_name character varying (50)	last_name character varying (50)	email character varying (100)	phone character varying (15)
1	2	Robert	Smith	robert.smith@email.com	9123456780

Step 3: Calculate Total Sales for the Current Month

This query calculates the total sales amount for the current month by summing up the total_amount from the orders table. It filters the orders by the current year and month.

Query Query History

```
    first_name,
    last_name,
    email,
    phone
FROM customers
WHERE last_name = 'Smith';  -- Replace 'Smith' with the last name you're searching for.

-- 3. Calculate total sales for the current month
SELECT
    SUM(total_amount) AS total_sales
FROM orders
WHERE EXTRACT(MONTH FROM order_date) = EXTRACT(MONTH FROM CURRENT_DATE)
  AND EXTRACT(YEAR FROM order_date) = EXTRACT(YEAR FROM CURRENT_DATE);
```

Data Output Messages Notifications

Showing rows:

	total_sales numeric
1	1997.00

Step 4: Update Stock When a Book is Sold

This query decreases the stock quantity of a book when it is sold. For example, in this case, the stock of book with book_id = 1 is reduced by 1.

```
FROM customers
WHERE last_name = 'Smith';  -- Replace 'Smith' with the last name you're searching for.

-- 3. Calculate total sales for the current month
SELECT
    SUM(total_amount) AS total_sales
FROM orders
WHERE EXTRACT(MONTH FROM order_date) = EXTRACT(MONTH FROM CURRENT_DATE)
  AND EXTRACT(YEAR FROM order_date) = EXTRACT(YEAR FROM CURRENT_DATE);

  -- 4. Update stock when a book is sold
UPDATE books
SET stock_quantity = stock_quantity - 1
WHERE book_id = 1;  -- Example: Decrease stock of book ID 1 by 1
```

Data Output | Messages | Notifications

```
UPDATE 1

Query returned successfully in 152 msec.
```

Step 5: List All Orders for a Specific Customer with Price in ₹ Symbol

This query lists all orders placed by a specific customer (in this case, customer with customer_id = 1).

```
WHERE book_id = 1;  -- Example: Decrease stock of book ID 1 by 1

-- 5. List all orders for a specific customer with price in ₹ symbol
SELECT
    o.order_id,
    o.order_date,
    o.total_amount,
    oi.book_id,
    oi.quantity,
    '₹' || oi.price AS price  -- Adds ₹ symbol before the price
FROM orders o
JOIN order_items oi ON o.order_id = oi.order_id
WHERE o.customer_id = 1;  -- Replace '1' with the customer_id you're interested in
```

Data Output | Messages | Notifications

Showing rows: 1 to 2 | Page No: 1

	order_id integer	order_date date	total_amount numeric (10,2)	book_id integer	quantity integer	price text
1	1	2025-10-10	1198.00	1	1	₹599.00
2	1	2025-10-10	1198.00	4	2	₹699.00

Conclusion:

The assignment involved setting up a simple database for a bookstore using PostgreSQL. I successfully created and populated the tables with sample data for books, customers, orders, and order items. Additionally, I wrote a series of operational queries to handle daily operations in a bookstore, such as checking stock, retrieving customer details, calculating total sales, updating stock after sales, and listing customer orders. This exercise helped me apply SQL concepts and improve my understanding of relational databases.

Screenshots and Results:

The document will also include screenshots of the PostgreSQL interface with successful queries executed, along with the resulting output of each query. The screenshots will show the creation and verification of tables, as well as the output of various operational queries that were executed to validate the implementation.